THESIS DEFENSE REFERENCE · PALAWAN STATE UNIVERSITY – RIZAL CAMPUS

ScholarHub — Thesis Defense Guide

Project: ScholarHub: An Online Scholarship Monitoring System for Palawan State University – Rizal Campus

Document purpose: A complete reference for explaining, justifying and defending the project before a thesis panel.

Prepared: September 2026

> **How to use this document.** Part I is what you say when someone asks "what is your project?" Part II is what you say when they ask "how does it work?" Part III is "why does it matter?" Part IV covers how it was built and tested. Part V is a demo script. Part VI is the largest section: the questions a panel is likely to ask, with model answers. Part VII is honest about limitations. Part VIII is delivery advice. The appendices are a cheat sheet you can print and keep on the table.
>
> Anything marked [FILL IN] is something only you can supply (your name, your adviser, your respondents, your survey figures). Do not present this document with those markers still in it.

PART I — WHAT THE PROJECT IS

1.1 Title and one-line description

ScholarHub is a web-based scholarship monitoring system that lets students of PSU Rizal Campus discover scholarships, apply online with their requirements, and track the result — while giving the Scholarship Office one place to post programs, review applicants, record academic standing, generate reports and keep an audit trail of every decision.

1.2 The 30-second version (elevator pitch)

> "Scholarship processing on campus is mostly manual: printed forms, folders of requirements, and students walking to the office to ask 'is my application approved yet?' ScholarHub moves that whole cycle online. Students register with a verified email, browse open scholarships, upload their documents, and see their status in real time. The Scholarship Office reviews applicants, approves or rejects with remarks, and the system takes care of the rest — slot counting, notifications, email updates, reports, analytics, and a log of who decided what. It is built with React, Node/Express and PostgreSQL, deployed as a serverless application, and its core rules are covered by 49 automated tests."

1.3 The 2-minute version

ScholarHub has two kinds of users.

Students create an account, confirm their email address, and see the list of scholarships that are currently open. Each scholarship shows its benefits, qualifications, requirements, remaining slots and deadline. A student applies by filling in a form and uploading the required documents (PDF, Word or image files). After submitting, the student can track the application, and — while it is still pending — withdraw it or replace a document that was unreadable. When a coordinator decides, the student is notified in the app and by email. The student dashboard also shows their standing: eligibility score, GPA trend across terms, document completion, estimated award, and a renewal outlook.

Admins (Scholarship Office coordinators) manage the scholarship catalogue, post announcements to a chosen audience with a publish/expiry window, review applications and their attached files, approve or reject with remarks, and record each applicant's academic facts (term, program, GPA, award amount, disbursement date, requirement checklist). They have an analytics dashboard with filters (academic year, semester, program, scholarship, status, type), ten standing reports with CSV export and print layout, an account-management page for coordinators and students, and an audit log that records every decision, deletion and account change.

Underneath, the system enforces the office's rules on the server so they cannot be bypassed: a closed or expired scholarship accepts no applications; a student can have only one live application per scholarship; a slot is consumed only when an application is approved (never on submission); and two coordinators cannot approve the same last slot at the same time.

1.4 Background of the study

Scholarship programs are one of the main ways a state university keeps students enrolled. On a campus such as PSU Rizal, the Scholarship Office handles several programs at once, each with its own requirements, slot count and deadline, and each drawing applications from students across programs and year levels.

[FILL IN: describe the current process at the Scholarship Office as you observed it — e.g. how applications are submitted today, how records are kept (logbook, Excel, folders), how students learn their result, how reports are prepared for the administration. Cite your interview or observation date.]

A manual process of this kind has predictable weaknesses:

- **Information is scattered.** Announcements are on a bulletin board or a Facebook page; forms are on paper; the list of applicants is in a spreadsheet; requirements are in folders. Nothing links them.
- **Students have no visibility.** The only way to know an application's status is to ask in person.
- **Counting is error-prone.** Slot counts, applicant totals and per-program figures are recomputed by hand each time a report is requested, and "applicants" and "applications" are easily confused (a renewing scholar counted three times).

- **There is no record of decisions.** Who approved which application, and when, is not reliably recorded.
- **Renewal risk is invisible.** Nobody can see, before a term ends, which scholars are drifting toward the GPA floor.

ScholarHub was developed to address these weaknesses with a single system of record.

1.5 Statement of the problem

General problem. The scholarship application and monitoring process at PSU Rizal Campus is manual, fragmented and opaque, which causes delays, inconsistent records and a lack of timely information for both students and the Scholarship Office.

Specific problems the system answers:

1. How can students be given a single, always-available place to find open scholarships and their requirements?
2. How can applications and supporting documents be submitted and stored without paper, while remaining secure and private?
3. How can students be informed of their application status without visiting the office?
4. How can the Scholarship Office review, decide on and record applications consistently, with the office's rules enforced automatically?
5. How can accurate reports and analytics (by program, status, term, scholarship, and new vs. renewal) be produced on demand instead of recomputed by hand?
6. How can every decision and change be traced to the person who made it?
7. How can academic standing be used to give students and coordinators an early, explainable view of eligibility and renewal risk?

1.6 Objectives

General objective. To design, develop and evaluate a web-based scholarship monitoring system for PSU Rizal Campus that centralises scholarship information, application processing, status tracking, reporting and analytics.

Specific objectives. To develop a system that can:

1. Allow students to register with a verified campus email, browse scholarships, apply online and upload requirement documents.
2. Allow students to track their application status, receive notifications and email updates, and withdraw or amend a pending application.
3. Allow the Scholarship Office to manage the scholarship catalogue, announcements and user accounts.

4. Allow coordinators to review applicants and their documents, approve or reject with remarks, and record academic records (term, GPA, award, disbursement, requirement checklist).
5. Enforce the office's business rules on the server: deadlines, one live application per scholarship, slot consumption only on approval, and concurrency-safe approval of the last slot.
6. Provide an analytics dashboard, ten standing reports with CSV/print export, and a per-student eligibility, GPA-trend and renewal-outlook view.
7. Keep an audit trail of every administrative action.
8. Evaluate the system's acceptability using **[FILL IN: e.g. ISO/IEC 25010 criteria — functional suitability, reliability, usability, security — with N respondents]**.

1.7 Significance of the study / beneficiaries

Beneficiary	What they gain
Students	One place to find scholarships; apply from anywhere; see status instantly; get email and in-app notifications; correct a mistaken submission themselves; see their own eligibility and GPA standing.
Scholarship Office / coordinators	A single queue of applications with documents attached; consistent decisions with remarks; automatic slot accounting; reports and charts in seconds; an audit log; account management without IT support.
Campus administration	Accurate, filterable figures on applicants, recipients, disbursement and renewal; evidence of who decided what; a system that runs on free-tier cloud services.
Future researchers / developers	A documented, tested codebase with the business rules and prediction logic isolated in pure modules that can be extended (e.g. a trained renewal model once outcome data exists).

1.8 Scope and delimitation

In scope

- Two roles: Student and Admin (coordinator).
- Scholarship catalogue with slots, deadline, status, benefits, qualifications and requirements.
- Online application with up to 10 document uploads (10 MB each; PDF, Word, JPG, PNG, HEIC, WEBP).
- Application lifecycle: Pending Review → Approved / Rejected, plus student-initiated Withdrawn.
- Email verification, password reset, account deactivation, role management.
- Announcements with audience and publish/expiry window.
- Derived notification feed for both roles.

- Admin analytics (summary counts, eight charts), ten reports with CSV and print/PDF, student analytics (eligibility score, GPA trend, renewal outlook, estimated award and disbursement date).
- Audit log of all administrative and student-record actions.
- Rate limiting on authentication endpoints.
- Deployment on Netlify (front end + serverless API) and Supabase (PostgreSQL + file storage).

Out of scope / delimitations

- No online payment or actual fund disbursement; the system records disbursement, it does not perform it.
- No integration with the university's registrar or grading system; GPA is entered by the coordinator.
- The renewal prediction is a transparent, rule-based weighted model, not a machine-learning model trained on historical outcomes (there is no historical outcome data yet).
- Single campus, single office; multi-campus tenancy is future work.
- Mobile-responsive web only; no native mobile app.

1.9 Definition of terms

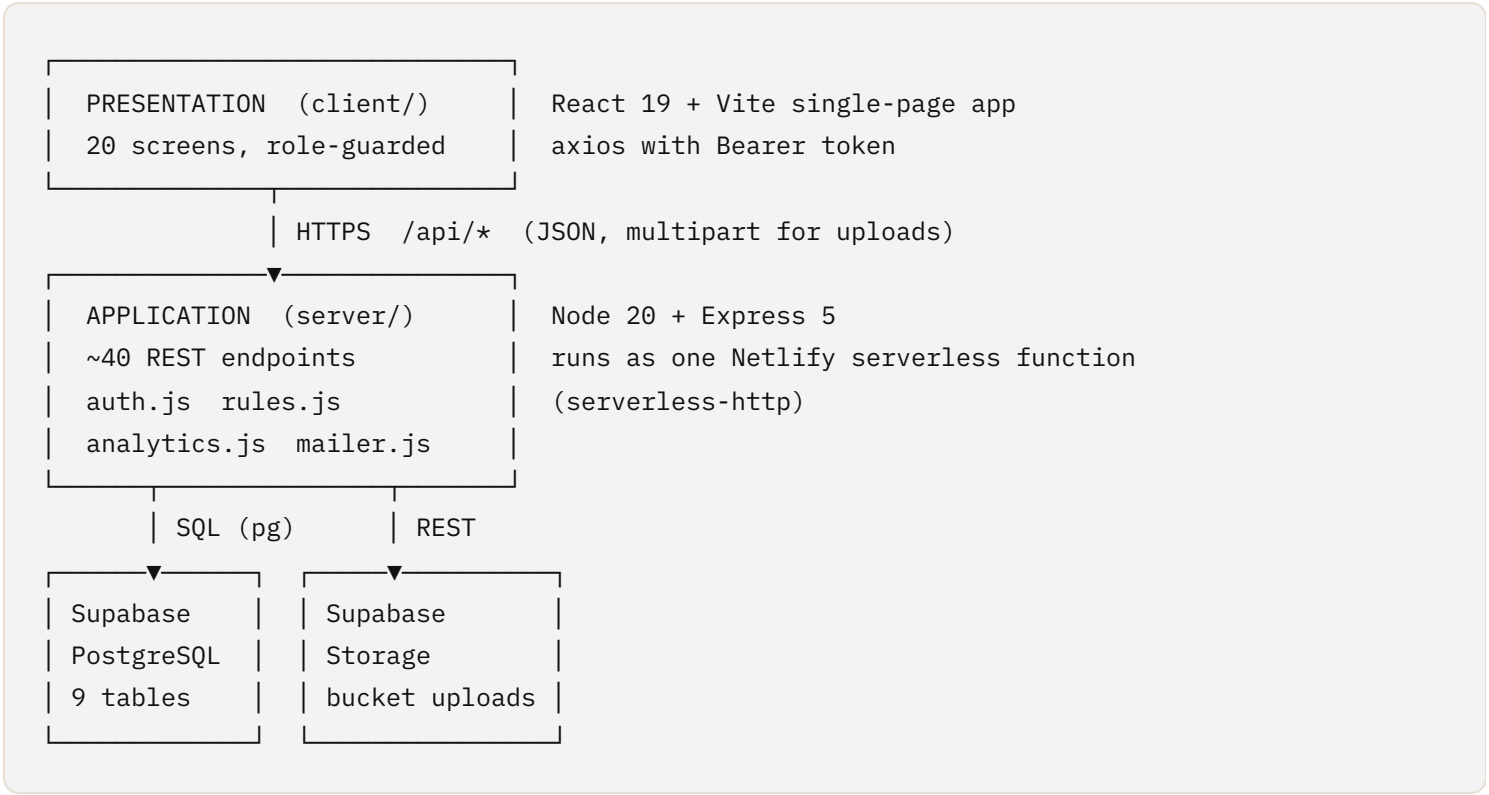
Term	Meaning in this project
Scholarship	A program posted by the office with benefits, qualifications, requirements, a slot count, a deadline and a status (Open/Closed).
Application	One form filed by one student for one scholarship in one term. A renewing scholar has several applications over time.
Applicant	A distinct person. Counted once regardless of how many applications they filed.
Slot	One award position. Consumed when an application is approved, returned when an approval is reversed or deleted.
Pending Review	Initial status; the only state in which a student may withdraw or change documents.
Withdrawn	Status set by the student. Never held a slot; blocks nothing; the student may apply again.
Renewal	An application by a student who has previously applied for the same scholarship.
Eligibility score	A 0–100 weighted score from academic standing (40), requirements submitted (30), application standing (20) and scholarship history (10).

Term	Meaning in this project
Renewal outlook	The eligibility score adjusted for GPA against the retention minimum, grade trend, document completion and past renewals, with a stated confidence and basis.
Audit log	An append-only record of who changed what and when.
JWT	JSON Web Token; the signed session token issued at login and verified on every API request.
Serverless function	The Express API running on demand inside Netlify rather than on a permanently running server.

PART II — HOW THE SYSTEM WORKS

2.1 Architecture

ScholarHub is a three-tier web application:



Key points to say out loud:

- The **client never talks to the database**. Every read and write goes through the API, which checks the caller's role from the signed token.
- The **server is stateless**. It holds no sessions in memory, which is what allows it to run as a serverless function that Netlify starts on demand.
- **Uploads never touch the server's disk.** Files stream to Supabase Storage and are served back only through an authorised endpoint.
- **Email** (Brevo or Resend) is optional: if no key is configured the mailer becomes a no-op and the system still works.

2.2 Technology stack and why each was chosen

Layer	Technology	Why
Front end	**React 19**, **React Router 7**, **Vite 8**	Component model suits a 20-screen app with shared UI kit; Vite gives fast builds; React is the most widely documented choice, which matters for future maintainers.
HTTP client	**axios**	Interceptors attach the JWT to every request and redirect to `/login` on 401 in one place.
Charts	**Hand-rolled SVG** (`charts.jsx`, 14 components)	No 100 KB charting dependency; colours come from the same CSS variables as the app so dark mode needs no chart code; accessibility rules (legend, table view, values never carried by colour alone) enforced once.
Back end	**Node 20**, **Express 5**	Same language as the front end; Express is minimal and well understood; Express 5 has native async error handling.
Auth	**jsonwebtoken**, Node **crypto.scrypt**	JWT makes the API stateless; scrypt is a memory-hard password hash built into Node (no extra dependency).
Uploads	**multer** (memory) → **Supabase Storage**	Serverless functions have no persistent disk.
Database	**PostgreSQL** on **Supabase**	Relational data with foreign keys; transactions for slot accounting; free tier; SQL editor for setup.
Email	**Brevo / Resend** via `mailer.js`	Password reset, verification and status emails; provider swappable.
Hosting	**Netlify** (static site + function)	Zero-cost hosting; single deploy for both halves via `netlify.toml`.
Testing	**node:test** (built-in)	49 unit tests on `auth.js`, `rules.js`, `analytics.js` with no database required.
Lint	**ESLint 10** + react-hooks plugin	Catches hook misuse and dead code before build.

2.3 Database design (9 tables)

Table	Purpose	Notable design decision
`users`	Students and admins	Never deleted — `is_active` + `deactivated_at` instead, so applications, awards and audit entries never orphan. `email_verified_at` NULL = unverified.
`scholarships`	The catalogue	`available_slots`, `deadline`, `status`.
`applications`	One row per form	`scholarship_id … ON DELETE SET NULL` keeps history if a scholarship is removed. Carries `academic_year`, `semester`, `program`, `application_type`, `gpa`, `award_amount`, `disbursed_at`, `documents_json` so a student's GPA history is simply their ordered applications.
`announcements`	Office notices	`audience` (All/Students/Admins), `publish_at`, `expires_at` (both nullable = post now, leave up).
`password_reset_tokens`	One-time reset links	Hashed token, expiry, single use.
`email_verification_tokens`	Confirmation links	Same pattern.
`rate_limits`	Throttle counters	Keyed by action + email + IP.
`notification_reads`	Which derived notification a user has seen	Only the *read state* is stored; the feed itself is derived from announcements and applications.
`audit_log`	Who changed what	`actor_email` denormalised on purpose so the entry survives renames/deactivation; deletions name the student and scholarship, not just an id.

Why academic data lives on `applications` and not a separate `academic_records` table: the application is the unit the office already works in. The coordinator records the GPA and award in the same place they approve or reject. A student's history reads as the ordered list of their own applications. A separate table would add a join and a second place to get out of step.

2.4 Roles and features

Feature	Student	Admin
Register, verify email, sign in, reset password	✓	✓
Edit profile and avatar	✓	✓
Browse open scholarships	✓	✓
Submit application with documents	✓	—
Track status; withdraw / add / remove documents while pending	✓	—
Notifications feed (in-app) + email on decision	✓	✓ (review queue, withdrawals)
Student dashboard & full analytics (eligibility, GPA trend, renewal outlook, award estimate)	✓	—
Create / edit / delete scholarships	—	✓
Create / edit / delete announcements with audience & window	—	✓
Review applicants, view files, approve / reject with remarks, delete	—	✓
Record academic facts (term, program, GPA, award, disbursement, checklist)	—	✓
Admin analytics dashboard (summary counts, 8 charts, 6 filters)	—	✓
Ten reports, CSV export, print/PDF	—	✓
Manage accounts (add, change role, deactivate, send password link)	—	✓
Audit log	—	✓

2.5 Screens (20)

Route	Screen	Role
`/`	Home	Public

Route	Screen	Role
`/login`, `/register`, `/forgot-password`, `/reset-password`, `/verify-email`	Authentication	Public
`/scholarships`	Scholarship catalogue	Student
`/application-form`	Apply	Student
`/track-status`	Track applications; withdraw / manage documents	Student
`/student-dashboard`	Condensed standing	Student
`/analytics`	Full academic/eligibility record	Student
`/notifications`	Feed	Both
`/admin-dashboard`	Analytics dashboard	Admin
`/manage-scholarship`	Catalogue CRUD	Admin
`/manage-applicants`	Review queue + Edit record	Admin
`/reports`	Ten reports	Admin
`/announcements`	Announcement CRUD	Admin
`/manage-users`	Accounts	Admin
`/audit-log`	Audit trail	Admin
`*`	Not found	—

2.6 Core workflow: the life of an application

1. **Register** → verification email → confirm. (Sign-in is allowed before confirming; *applying* is not.)
2. **Browse** → scholarships filtered to Open and not past deadline (deadline compared in Manila time).
3. **Apply** → form + up to 10 files. Server checks: scholarship open, deadline not passed, email verified, no live application already. The application is stamped with its academic year, semester (from the academic calendar), program, and type (New/Renewal).
4. **Pending Review** → student may withdraw, add or remove documents. Admin sees it in the review queue and in their notifications.

5. **Decision** → admin approves or rejects with remarks. On **Approved**, one slot is claimed atomically (`UPDATE … WHERE available_slots > 0`); if none remain the approval is refused with 409. Audit entry written. Student notified in-app and by email.
6. **Record** → admin enters GPA, award, disbursement date and requirement checklist via *Edit record*. This feeds analytics.
7. **Reversal** → if an approval is changed to Rejected/Pending, or an approved application is deleted, the slot returns.
8. **Next term** → the student applies again; because an approval from an earlier term blocks nothing, this becomes a Renewal application and extends the GPA trend.

2.7 Business rules enforced on the server (`rules.js`)

These are pure functions — no database, no Express — so they are unit-tested directly. Each takes a row and returns either `null` (allowed) or `{ status, message }` (refused).

Rule	Why
A closed or expired scholarship takes no applications (409).	The client hides the card, but the API accepts ids; the server must refuse.
One live application per student per scholarship: refused if one is pending, or one is approved *for the same term*. Rejected and Withdrawn block nothing; an earlier-term approval blocks nothing.	Prevents duplicates while keeping renewals and re-application after rejection possible.
A slot is an award, not an applicant: consumed only on Approved; returned on reversal or deletion; re-approving does not take a second slot.	Submission-time slot charging made the counter drift low by the number of never-approved applicants.
Concurrency: last slot claimed with a single conditional UPDATE.	Two coordinators approving simultaneously cannot both succeed.
Withdraw / add / remove documents only in Pending Review (409 otherwise).	Once ruled, the file set is the evidence the ruling was made on.
Withdrawal never moves a slot.	A pending application never held one.
Ownership checked from the token's `user_id`, not from the request.	`requireRole("Student")` says the caller is *a* student, not *this* student.
An admin cannot change their own role or deactivate themselves; the last active admin cannot be demoted or deactivated by anyone.	Both are the same failure: an office locked out of its own system.

Rule	Why
An announcement cannot expire before it is published; students receive only rows inside their window and audience (filtered in SQL).	A scheduled announcement must never reach the browser early.

2.8 Security design

Concern	Mechanism
Passwords	scrypt hashes (`scrypt$N$r$p$salt$hash`); legacy plaintext accounts upgraded on next successful login.
Sessions	JWT signed with `JWT_SECRET` , 8-hour expiry, sent as `Authorization: Bearer` ; 401 clears the client session.
Authorisation	Every endpoint re-checks the role from the token. The React route guards only decide what to render — editing `localStorage` changes the menu, not the data.
Identity	Never from the body or query string; a student's applications are looked up from the token.
Uploads	Allow-list of PDF/Word/JPG/PNG/HEIC/WEBP, 10 files × 10 MB. Served via `/api/applications/:id/file` with `X-Content-Type-Options: nosniff` ; anything outside the allow-list is sent as an octet-stream attachment so HTML/SVG can never execute in the app's origin.
File links	Browsers cannot attach headers to `<a href>` , so the client requests a 5-minute, single-application file token first.
Throttling	`rate_limits` table keyed by action + email + IP: login 8/15 min (failures), register 5/h, password-forgot 5/h, password-reset 10/h (failures), email-verify 8/h. A counter that cannot be read never blocks the action.
Email verification	Gate is on *submitting an application*, not on sign-in, so a typo can be corrected. Optional `ALLOWED_EMAIL_DOMAINS` restricts self-registration to campus addresses.
Admin-created accounts	Admin never chooses a password; owner receives a set-password link at *their* address.
CORS	Same-origin by default; extra origins via `CORS_ALLOWED_ORIGINS` .
Body size	JSON limited to 100 KB.

Concern	Mechanism
Audit	Every administrative write leaves an entry; writing the entry never fails the action it describes.

2.9 Analytics and prediction model (`analytics.js`)

Two principles are encoded, and you should state them before any number:

1. **A prediction is never presented as a decision.** Every predictive function returns the `basis` it was computed from and the UI prints it beside the number.
2. **A missing input lowers confidence; it never becomes an invented value.** A student with no grades gets "not enough history", not a fabricated 92 %.

Figure	Computation
Eligibility score (0–100)	Academic standing 40 + requirements submitted 30 + application standing 20 + scholarship history 10. Missing GPA scores the academic block at its midpoint and says so.
Renewal outlook	Eligibility score adjusted for GPA vs. retention minimum, grade trend (improving/declining), document completion and past renewals. Confidence drops with fewer than two terms of grades.
Estimated award	Amount recorded on the application → else the range this scholarship has actually paid → else a peso figure parsed from its published benefits → else nothing.
Disbursement date	Median approval-to-release gap for this scholarship applied to the approval date, ± the spread; with no history, the office's standard 30-day window.
Document completion	Coordinator's checklist when one exists, otherwise uploads counted against the scholarship's requirement list; returned per term to show whether paperwork improves.
GPA scale	`GPA_SCALE` = 4.00 scale, higher is better, 2.00 retention minimum. A campus on the 1.00–5.00 scale changes four values; every comparison goes through `gpaQuality()` / `isBetterGpa()`.

Applicants vs. applications. The dashboard shows both. A scholar who renews across three terms is one applicant and three applications; conflating them is the usual way a scholarship report overstates its reach.

2.10 Admin analytics dashboard and reports

Dashboard — filters: academic year, semester, program, scholarship, status, application type. Two groups of summary counts — Students (who applied, qualified, not qualified, awaiting decision) and Applications (received, approved, rejected, pending) — and eight charts: applications over time (12 months, submitted vs approved), by program, by status, qualified vs not qualified, per semester, recipients per program, new vs renewal, requirements on file. Every figure is computed from the same filtered slice in one request so no chart can disagree with another.

Ten reports (same filters, CSV export, print/PDF layout): Applications summary · Total applicants · Qualified applicants · Rejected applicants · Applicants by program · Applicants by status · Scholarship recipients · Renewal statistics · GPA performance · Scholarship disbursement · Document completion.

2.11 Notifications and announcements

- **The feed is derived, not stored.** An item is an announcement or a decision on an application — both already rows. Only *read state* is stored (`notification_reads`), keyed by id *and* status (`application-88-Approved`) so a later decision arrives as a new unread item.
- One endpoint, `GET /api/notifications` , serves the navbar bell and the notifications page for both roles, so they can never disagree.
- Announcements carry audience, publish time and expiry; students' window is filtered in SQL; admins see all rows with Live/Scheduled/Expired tags.

2.12 Audit trail

Every coordinator write (decisions, deletions, academic records, scholarship and announcement changes, account changes) and every student withdrawal/document change is appended to `audit_log` . Two deliberate decisions: writing the entry never fails the action (an award granted stays granted even if the log insert fails — the failure is logged instead), and deletions record the student and scholarship names because the row they refer to no longer exists.

2.13 Operational tooling

- `npm run db:seed-demo -- --confirm` / `--remove` — reversible, tagged demo data for the dashboards; refuses to run over real data.
- `npm run db:repair-slots` — one-time reconstruction of slot counts after the slot rule changed.
- `npm run db:setup` — adds new columns to an existing database.
- `npm start` — runs API (:5001) and client (:5173) in one terminal.

PART III — WHY IT MATTERS: IMPORTANCE AND BENEFITS

3.1 Problem → solution map

Problem in the manual process	ScholarHub answer
Students don't know which scholarships are open or what they need	Catalogue with benefits, qualifications, requirements, slots, deadline; closed/expired hidden automatically
Paper forms and physical folders	Online form; documents in cloud storage; accessible to the coordinator from the applicant's row
"Is my application approved yet?"	Track Status page, in-app feed, email on decision
A wrong submission needs the office to delete it	Student withdraws or replaces documents while pending
Slot counts drift	Slot consumed only on approval, returned on reversal, concurrency-safe
Duplicate applications	One live application per scholarship per term, enforced server-side
Reports recomputed by hand	Ten reports, six filters, CSV and print, from one consistent data slice
Applicants vs applications confusion	Both counted and labelled separately
No record of who decided	Audit log with actor, action, target, timestamp
Renewal risk invisible	GPA trend, eligibility score, renewal outlook with basis
Announcements visible forever to everyone	Audience + publish/expiry window + delete
Only one admin account ever	Manage Accounts: add coordinators, deactivate graduates, send password links
Decisions emailed to unverified addresses	Email verification gate before applying

3.2 Benefits by stakeholder

For students

- 24/7 access from any device; no queueing at the office.
- Transparency: status, remarks, and their own eligibility figures with the reasons shown.
- Self-service correction (withdraw, replace a blurry scan).
- Fair play: the same rules apply to everyone because they are enforced by the server, not by whoever is at the desk.

For the Scholarship Office

- One queue, one record per application, documents attached.
- Time saved on counting, filtering and report preparation.
- Accountability and defensibility: every decision has an actor and timestamp.
- Early warning on scholars near the GPA floor.
- No IT ticket needed to add a coordinator or reset a student's password.

For the institution

- Reliable figures for planning and for reporting to funders.
- Near-zero hosting cost (Netlify + Supabase free tiers).
- A maintained, tested codebase that a future student team can extend.

3.3 Manual vs ScholarHub comparison

Criterion	Manual	ScholarHub
Where to apply	Office, office hours	Anywhere, any time
Status inquiry	Walk-in / call	Real-time page + notification + email
Document storage	Folders	Cloud storage, per application, access-controlled
Slot tracking	Manual tally	Automatic, transactional
Duplicate prevention	Coordinator's memory	Enforced rule
Report on applicants by program	Hours	Seconds, exportable
Decision record	Often none	Full audit log

Criterion	Manual	ScholarHub
Renewal risk	Not visible	GPA trend + outlook
Adding a coordinator	Not possible in old system	Manage Accounts

PART IV — HOW IT WAS BUILT AND TESTED

4.1 Development methodology

[FILL IN: state the SDLC model your adviser expects. The project's history fits an iterative/Agile approach — features were added in increments (authentication hardening → application rules → analytics → accounts → announcements → notifications → audit trail), each with its own tests and documentation.] Recommended phrasing:

> "We followed an iterative, incremental model. Each iteration took one workflow — for example, the application rules — from requirement to implementation to automated test to documentation. This let us validate early with the Scholarship Office and change requirements without rewriting the system."

Suggested phases to describe: (1) requirements gathering with the Scholarship Office [FILL IN dates]; (2) design — data model, roles, screens; (3) implementation in iterations; (4) testing — unit, integration/manual, user acceptance; (5) deployment on Netlify/Supabase; (6) evaluation [FILL IN instrument, respondents, results].

4.2 Testing

Level	What	Evidence
Unit	`auth.test.js` (hashing, token signing, file-token scoping), `rules.test.js` (every refusal rule), `analytics.test.js` (scoring and prediction)	**49 tests, 49 passing**, run with `npm test` in ~140 ms, no database required
Static	ESLint with react-hooks rules; production build	`npm run lint`, `npm run build` clean
Functional / manual	Every screen and workflow exercised in both roles; demo seed for analytics	[FILL IN: test-case table or checklist with results]
Concurrency	Last-slot approval by two admins	Conditional UPDATE; second returns 409 "scholarship is full"
Security	Role escalation via localStorage; identity via query string; unsafe file types	All refused by the server

Level	What	Evidence
User acceptance / evaluation	[FILL IN: e.g. ISO/IEC 25010 questionnaire, N students + N staff, Likert 1–5, mean per criterion]	[FILL IN results]

Why the rules are testable: `rules.js` and `analytics.js` are pure — they import neither Express nor `pg`. The decisions that matter most (does a slot move? is this renewal a duplicate? can the last admin be deactivated?) used to live inside request handlers and could only be tested by standing up Postgres and posting a multipart form; extracting them is what made 49 tests possible.

4.3 Deployment

- **Front end**: `client/dist` built by Vite, served by Netlify's CDN.
- **API**: the Express app wrapped with `serverless-http` and deployed as one Netlify function; handles `/api/` *and* `/uploads/`.
- **Database**: Supabase PostgreSQL via the transaction pooler (port 6543), suited to short-lived serverless connections.
- **Files**: Supabase Storage bucket `uploads`.
- **Configuration**: all secrets via environment variables (`DATABASE_URL`, `JWT_SECRET`, `SUPABASE_*`, mail key, `ADMIN_EMAIL` / `ADMIN_PASSWORD` for first run only).

4.4 Project size (for the "how big is this?" question)

- ~28,800 lines of application code across client, server and schema.
- 20 screens, ~40 API endpoints, 9 tables, 14 chart components, 49 unit tests.

PART V — DEMO SCRIPT (10–12 minutes)

Have two browsers open (or one normal + one private window): one signed in as a student, one as an admin. Seed demo data beforehand (`npm run db:seed-demo -- --confirm`) so charts are populated.

#	Step	What to say
1	Home page	"Public landing page; the three cards summarise the student journey."
2	Register a student, show the verification email / `/verify-email`	"Sign-in works before confirming; applying does not — so a typo can be fixed without a support ticket."
3	Scholarships page	"Only open, not-yet-expired programs appear. Slots, deadline, requirements are all here."
4	Apply, upload two files	"Up to 10 files, allow-listed types. Files go to cloud storage, never the server disk."
5	Track Status	"Pending — the student can still withdraw or replace a file. Watch these controls disappear after the decision."
6	Admin → Manage Applicants → open files → Approve with remarks	"Slot count drops by one *now*, not at submission. Approval writes an audit entry and emails the student."
7	Back to student → notification bell + Track Status	"Status updated; controls gone; the file set is now frozen evidence."
8	Admin → Edit record: term, GPA, award, checklist	"This is the only source of GPA. Students never self-report a grade."
9	Student → Dashboard → Analytics	"Eligibility score with its four parts; GPA trend; renewal outlook *with the basis printed beside it*."
10	Admin → Dashboard, change a filter	"Every chart moves together because they are computed from one slice."
11	Reports → export CSV → print view	"Ten standing reports; this is what goes to the administration."
12	Announcements: schedule one for tomorrow	"Student side: not visible yet — filtered in SQL, never reaches the browser."

#	Step	What to say
13	Manage Accounts → try to deactivate yourself	"Refused. The last active admin cannot be removed either."
14	Audit Log	"Everything we just did, with who and when."
15	(Optional) terminal: `npm test`	"49 tests on the rules and the prediction model, no database needed."

Rehearse the failure paths — they impress more than the happy path: apply twice (409), approve when slots are 0 (409), edit localStorage role to Admin and hit an admin endpoint (403).

PART VI — ANTICIPATED QUESTIONS AND MODEL ANSWERS

A. General / conceptual

Q1 In one sentence, what is ScholarHub?

A web-based system that centralises scholarship posting, online application, status tracking, review, reporting, analytics and auditing for the Scholarship Office and students of PSU Rizal Campus.

Q2 What problem does it solve that a spreadsheet or Google Forms would not?

Google Forms collects submissions; it does not enforce deadlines per scholarship, block duplicate applications, count slots transactionally, let students track and amend their own submission, notify on decision, restrict file access by owner, produce filtered reports, or record who approved what. Those rules live in our server and are tested.

Q3 Who are the users?

Two roles: Students (apply, track, view their standing) and Admins/coordinators (manage catalogue, review, record academics, report, manage accounts, audit).

Q4 Why "monitoring" system and not just "application" system?

Because the value is after submission: tracking, slot accounting, academic records, GPA trend, renewal outlook, reports and audit. Application intake is one of nine tables.

Q5 Is this an original idea?

Scholarship management systems exist; the contribution is a system tailored to this office's actual rules (slot = award, renewal handling, term-stamped applications), with an explainable analytics layer, an audit trail and a tested rule set, deployable at zero cost.

Q6 What is the "novelty" or contribution of the study?

(1) Encoding the office's business rules as pure, unit-tested functions rather than as UI behaviour; (2) an explainable eligibility/renewal model that prints its basis and never invents values; (3) a derived (not stored) notification design; (4) an audit trail whose entries survive deletions and renames; (5) a serverless deployment pattern for a campus system.

B. Requirements and methodology

Q7 How did you gather requirements?

[FILL IN: interview/observation with the Scholarship Office, dates, who; documents reviewed (forms, requirement lists).] From that we derived the roles, the four application rules, the reporting needs (by program, status, term, recipients, renewal, GPA, disbursement, documents) and the requirement to know who decided.

Q8 What SDLC did you use and why?

Iterative/incremental (Agile-style). Each iteration delivered one workflow end-to-end with tests and documentation, so feedback from the office could change the next iteration without rework. See §4.1.

Q9 How did you evaluate the system?

[FILL IN: instrument, criteria, respondents, sampling, results. If ISO/IEC 25010: functional suitability, performance efficiency, usability, reliability, security, maintainability.] Plus 49 automated tests and manual functional testing of every workflow.

Q10 What was your sample size and how was it chosen?

[FILL IN.]

C. Architecture and technology

Q11 Why React instead of plain HTML/PHP or Vue/Angular?

Twenty screens share a UI kit (stat tiles, filter bar, callouts, charts); a component model keeps that consistent. React has the largest ecosystem and documentation, which matters because a future student team will maintain it. Any of the three would work; React was the one the team could deliver and test fastest.

Q12 Why Node/Express rather than PHP/Laravel or Django?

One language across the stack; Express is small enough that every route is visible in one file; Node's built-in `crypto.scrypt` and `node:test` removed two dependencies; and Express wraps cleanly into a Netlify serverless function.

Q13 Why PostgreSQL and not MySQL or MongoDB?

The data is relational — applications point at users and scholarships, audit entries point at both — and slot accounting needs a transactional conditional UPDATE. Supabase provides managed Postgres with storage on a free tier. MongoDB would give up foreign keys and make the slot rule harder to guarantee.

Q14 Why serverless? Isn't that overkill for a campus?

The opposite: it is the cheapest and simplest to operate. There is no server to patch or keep running; Netlify starts the function on request. The trade-off is cold-start latency and no local disk, which is why uploads go to Supabase Storage and the API is stateless.

Q15 Why did you write your own charts instead of using Chart.js or Recharts?

Three reasons: the bundle stays ~100 KB lighter on a serverless deployment; the charts read colours from the same CSS variables as the app, so dark mode needs no chart-specific code; and accessibility rules (legend for 2+ series, a data table behind every chart, values never carried by colour alone) are enforced in one place.

Q16 How does the front end talk to the back end?

JSON over HTTPS to `/api/*`, with `Authorization: Bearer <JWT>`; multipart for uploads. axios interceptors attach the token and handle 401.

Q17 Where are uploaded files stored and who can see them?

Supabase Storage. They are not publicly reachable. `/api/applications/:id/file` checks that the caller is the owner or an admin; because a browser cannot add a header to a link, the client first obtains a 5-minute token scoped to that one application.

Q18 What happens if the email service is not configured?

`mailer.js` becomes a no-op; registration, verification (can be disabled with `REQUIRE_EMAIL_VERIFICATION=false`) and decisions still work; only the emails are skipped.

D. Security

Q19 How are passwords stored?

As scrypt hashes with a per-user salt, in the format `scrypt$N$r$p$salt$hash`. scrypt is memory-hard, which slows GPU brute force. Old plaintext accounts are upgraded on the next successful login.

Q20 What stops a student from changing their role to Admin in the browser?

Nothing stops them changing the *menu* — but every endpoint re-verifies the role from the signed JWT. Editing localStorage changes what React renders, not what the server allows. We can demonstrate: set role to Admin, call `/api/users`, get 403.

Q21 What stops a student from viewing another student's application?

Identity never comes from the request. `isApplicationOwner` compares the token's `user_id` with the row's; `requireRole("Student")` only proves the caller is *a* student.

Q22 Could someone upload a malicious file?

The allow-list accepts PDF, Word and common images only. Because the browser declares the type, that is the first gate, not the last: files are served with `X-Content-Type-Options: nosniff`, and anything outside the allow-list is sent as an octet-stream attachment, so HTML/SVG can never execute in the app's origin — including files uploaded before the allow-list existed.

Q23 How do you prevent brute-force login or email spamming?

A `rate_limits` table keyed by action + email + IP: login 8 failures per 15 min, register 5/h, password-forgot 5/h, password-reset 10 failures/h, email-verify 8/h. Keying on email *and* IP avoids both walking an address list from one machine and locking out a whole campus NAT.

Q24 Why does email verification not block login?

Because the gate exists to make sure decisions are mailed to the right person, and locking the account turns a typo into a support ticket. A student can sign in and fix the address; they cannot *apply* until it is confirmed (403 `EMAIL_NOT_VERIFIED`).

Q25 What about SQL injection / XSS?

All queries use parameterised statements through `pg`; user input is never concatenated into SQL. React escapes rendered text by default. Uploaded files are never rendered inline unless they are on the

image/PDF allow-list.

Q26 What is in the JWT and how long does it last?

User id, email and role, signed with `JWT_SECRET`; expires in 8 hours. There is no server-side session store, which is what allows stateless serverless operation.

Q27 Can an admin lock everyone out by mistake?

No. An admin cannot change their own role or deactivate themselves, and the last active admin cannot be demoted or deactivated by anyone. Both are enforced in `rules.js` and tested.

E. Business logic

Q28 When is a slot consumed?

Only when an application becomes Approved. Not on submission, not on withdrawal. Reversal or deletion of an approved application returns it. Re-approving an already-approved application does not take a second slot.

Q29 Two coordinators approve the last slot at the same second. What happens?

The claim is one SQL statement: `UPDATE scholarships SET available_slots = available_slots - 1 WHERE id = $1 AND available_slots > 0`. Only one can match; the other updates zero rows and is told the scholarship is full (409). No application-level lock is needed.

Q30 Can a student apply twice for the same scholarship?

Not while one is pending or approved for the same term. After rejection or withdrawal they can. An approval from an earlier term does not block — that is what makes renewals possible.

Q31 What is the difference between "applicant" and "application" on your dashboard?

Applicant = distinct person; application = one form in one term. A scholar renewing across three terms is one applicant and three applications. We show both so reach is not overstated.

Q32 Why can a student withdraw only while pending?

Because a pending application never held a slot, so withdrawal never has to move one. Allowing withdrawal after approval would return a slot on a path no coordinator ever sees.

Q33 Why is GPA entered by the coordinator and not the student?

A self-reported GPA feeding a renewal prediction would make the prediction worthless. The office already has the grade sheet; it records it in the same place it approves the application.

Q34 How is the deadline compared? What about time zones?

Against the Manila calendar date, not the database's UTC date, so a scholarship does not close eight hours early.

F. Analytics and prediction

Q35 How is the eligibility score computed?

Weighted 0–100: academic standing 40, requirements submitted 30, application standing 20, scholarship history 10. If GPA is missing, the academic block scores its midpoint and the UI says so. All in `analytics.js`, unit-tested.

Q36 How is the renewal outlook computed? Is it machine learning?

No — it is a transparent, rule-based weighted model: the eligibility score adjusted for GPA against the retention minimum, grade trend, document completion and past renewals. Confidence drops with fewer than two terms of grades. It is deliberately legible: a student who asks "why 24 %?" can be shown the same four numbers.

Q37 Has the prediction been validated?

Not against real outcomes, because no historical renewal data exists yet — the system is what will create it. This is stated as a limitation. Once two or three terms of outcomes exist, the weights can be calibrated or replaced by a trained model behind the same interface.

Q38 Why not use machine learning?

No labelled data; a black-box score on a student's scholarship would be unethical to present without explanation; and the office needs to be able to justify a figure to a student. The rule model is the right first step and generates the data an ML model would later need.

Q39 What if a student has no GPA recorded?

The system says "not enough history" and lowers confidence. It never invents a value.

Q40 How is the estimated award computed?

Recorded amount → else the range this scholarship has actually paid → else a peso figure parsed from the published benefits → else no number is shown.

Q41 How is the disbursement date estimated?

Median approval-to-release gap for that scholarship applied to the approval date, ± the spread; with no history, a standard 30-day window.

Q42 Why are all charts computed in Node from the raw rows instead of SQL GROUP BY?

So every chart on the page is computed from the same snapshot; independent GROUP BY queries could disagree under concurrent writes. It is capped at 20,000 rows per request and is the first thing to move into SQL if the table outgrows that.

Q43 Your GPA scale is 4.00; PSU uses 1.00–5.00. Is the system wrong?

`GPA_SCALE` is one constant with four values (min, max, retention minimum, direction); every comparison goes through `gpaQuality()` / `isBetterGpa()`. Switching scales is a four-value change with tests covering both directions.

G. Data and database

Q44 Why are users never deleted?

Applications, awards and audit entries point at them; deleting the row would orphan the record of who received what. Deactivation stops sign-in and keeps history whole.

Q45 What happens to applications when a scholarship is deleted?

`ON DELETE SET NULL` — the applications remain with their stored `scholarship_title`; only the link is blanked.

Q46 Why is the notification feed derived instead of stored?

Announcements and decisions are already rows; a notifications table would be a second copy to keep in sync. Only what cannot be derived — whether a user has *seen* an item — is stored.

Q47 Why is `actor_email` duplicated in `audit_log` when you have `actor_id` ?

So the entry still means something after a rename or deactivation: it records the address that held the account *at the time*.

Q48 How is the database set up and migrated?

`database/schema.sql` creates everything once; `npm run db:setup` adds new columns to an existing database. Migration is no longer run on every cold start (performance fix in the commit history).

H. Testing, quality, limitations

Q49 What tests do you have?

49 unit tests over three pure modules — hashing/tokens, the refusal rules, the scoring/prediction — passing in ~140 ms with no database. Plus ESLint, a production build, manual functional testing of every workflow, and **[FILL IN user evaluation]**.

Q50 Why unit tests only? Where are integration tests?

The highest-risk logic was deliberately extracted into pure modules so it could be tested without infrastructure. Endpoint-level tests against a test database are the natural next step and are listed as future work.

Q51 What are the limitations of the system?

See Part VII. The headline ones: prediction not yet validated against outcomes; single campus; polling (5 s) rather than push for notifications; analytics aggregated in Node up to 20 k rows; audit log has no retention policy; no registrar integration.

Q52 What would you do differently?

Split `Navbar.jsx` and `ApplicationForm.jsx` earlier; write endpoint tests alongside the unit tests; verify a sending domain for email from the start so messages avoid spam folders.

Q53 How scalable is it?

For one office it is well within limits: serverless scales horizontally; Postgres handles far more than a campus's applications; the known ceilings (20 k rows per analytics request, 500 rows per detail report, 50 items per notification feed) are documented with the fix for each.

Q54 How much does it cost to run?

Nothing at campus scale: Netlify and Supabase free tiers; Brevo's free email tier.

Q55 Is it accessible / mobile-friendly?

Responsive layout; every chart ships a legend and a data-table view; values are never carried by colour alone; line icons rather than emoji for consistent rendering; dark mode.

I. Curveballs

Q56 What if the internet is down at the office?

The system is unavailable, like any web system. Paper fallback remains possible; records are entered when connectivity returns. Offline-first is out of scope.

Q57 What if Supabase or Netlify shuts down?

Both are standard Postgres and Node — the schema and the Express app run unchanged on any Postgres host and any Node server (`npm run dev` runs it locally today).

Q58 Did you write all the code yourself?

[FILL IN honestly — team members, adviser input, any AI-assisted tooling used, and what you personally understand and can modify. Be ready to explain any file on request.]

Q59 Can this be used by other campuses?

Yes with configuration: institution name, GPA scale constants, `ALLOWED_EMAIL_DOMAINS`. Multi-campus in one deployment (tenancy) is future work.

Q60 What is the single most important design decision?

Keeping the rules the system *refuses on* in a pure, tested module rather than in the UI or scattered through handlers. Everything the panel might worry about — duplicates, slots, lockout — is answered by pointing at a test.

PART VII — LIMITATIONS AND FUTURE WORK (say these before the panel does)

Limitation	Why it exists	Future work
Renewal outlook not validated against real outcomes	No historical data yet	Calibrate weights or train a model after 2–3 terms
GPA entered manually by coordinator	No registrar API access	Registrar/grading-system integration
Notifications polled every 5 s, 50-item cap	Adequate for one office	Cursor pagination + push (SSE/WebSocket)
Analytics aggregated in Node (≤ 20 k rows)	Cross-chart consistency	Move to SQL/materialised views when needed
Detail reports show 500 most recent rows	Page says so	Server-side pagination
`audit_log` grows forever	Correct default for audit	Archiving policy
Edited announcement stays "read"	Typo fix should not re-notify	"Republish" action to reset read state
Emails from free sender may hit spam	No verified domain	Verify a campus domain in Brevo
Old secrets/uploads remain in git history	Pre-cleanup commits	`git filter-repo` before making the repo public
Single campus / single office	Scope	Multi-tenant support
No endpoint-level automated tests	Time	Supertest + test database
Large `Navbar.jsx` / `ApplicationForm.jsx`	Grew with features	Split into smaller components
No native mobile app	Scope	Responsive web covers it; PWA is a cheap next step

PART VIII — DEFENSE DELIVERY TIPS

1. **Open with the problem, not the technology.** Two sentences on the office's manual process, then the one-line description.
2. **Say the two analytics principles before any number.** "A prediction is never a decision" and "missing data lowers confidence, it never invents a value" pre-empt the hardest analytics questions.
3. **Show failure paths in the demo.** Duplicate application → 409; approve at zero slots → 409; role edit in localStorage → 403. Panels remember refusals more than success screens.
4. **Point at tests.** For any rule question: "That rule is in `rules.js` and there is a test for it." Have the test names visible.
5. **Name limitations yourself** (Part VII). A limitation you raise is maturity; one the panel raises is a weakness.
6. **Never say "the system cannot do that" alone.** Say "out of scope because X; it would be added by Y."
7. **When you don't know, say so and reason aloud.** "I'm not certain; I'd check `analyticsRoutes.js`, and my expectation is …" beats a guess.
8. **Keep answers to 30–60 seconds.** Answer, one reason, offer to show. Let the panel ask for more.
9. **Distinguish applicants from applications every time you cite a count.**
10. **Bring the cheat sheet (Appendix D)** and a printed copy of the schema.

APPENDIX A — API endpoints

Method & path	Role	Purpose
POST `/api/register`	Public	Create student account, send verification
POST `/api/login`	Public	Returns JWT
GET `/api/me`	Any	Current user
POST `/api/email/verify`, `/api/email/resend`	Student	Email confirmation
POST `/api/password/forgot`, `/reset`, `/change`	Public / Any	Password flows
PUT `/api/users/profile`, POST/PUT `/api/users/avatar`	Any	Profile
GET `/api/scholarships`	Any	Catalogue (students: open & not expired)
POST/PUT/DELETE `/api/scholarships[/:id]`	Admin	Catalogue CRUD
POST `/api/applications`	Student	Submit (multipart)
GET `/api/applications/student`	Student	Own applications
GET `/api/applications`	Admin	All applications
PUT `/api/applications/:id/status`	Admin	Approve / reject / pending
DELETE `/api/applications/:id`	Admin	Delete (returns slot if approved)
POST `/api/applications/:id/withdraw`	Student (owner)	Withdraw while pending
POST/DELETE `/api/applications/:id/documents[/:index]`	Student (owner)	Amend files while pending
GET `/api/applications/:id/file-token`, `/file`	Owner or Admin	Secure file access

Method & path	Role	Purpose
PUT `/api/applications/:id/academics`	Admin	Term, GPA, award, disbursement, checklist
GET `/api/announcements`, POST/PUT/DELETE	Any / Admin	Announcements
GET `/api/notifications`, POST `/read`, `/read-all`	Any	Derived feed and read state
GET `/api/admin/analytics`, `/dashboard-stats`, `/recent-activity`	Admin	Dashboard & reports data
GET `/api/student/analytics`	Student	Own standing
GET/POST/PUT/DELETE `/api/users[/:id]`, POST `/:id/password-link`	Admin	Account management (DELETE deactivates)
GET `/api/admin/audit-log`	Admin	Audit trail

APPENDIX B — Database tables

`users` · `scholarships` · `applications` · `announcements` · `password_reset_tokens` · `email_verification_tokens` · `rate_limits` · `notification_reads` · `audit_log`

APPENDIX C — Repository layout

```
client/src/
  api.js                      axios instance, token storage, 401 handling
  App.jsx                     routes and render-time role guards
  pages/                      one file per screen (20)
  components/Navbar.jsx       nav, profile menu, notifications bell
  components/charts.jsx       SVG chart kit (14 components)
  components/ui.jsx           stat tiles, filter bar, callouts, checklists
  components/AcademicRecordDialog.jsx  admin editor for term/GPA/award/docs
server/
  server.js                   Express app, routes, handlers
  auth.js                     scrypt hashing, JWT sign/verify
  rules.js                    the rules the app refuses on (pure)
  analytics.js                scoring and prediction (pure)
  analyticsRoutes.js          analytics, reports, academic-record endpoints
  mailer.js                   outbound email (no-op when unconfigured)
  storage.js                  Supabase Storage client
  *.test.js                   49 unit tests
  scripts/                    seed-demo-data, repair-slot-counts
database/schema.sql           full schema
netlify.toml                  build + function config
```

APPENDIX D — One-page cheat sheet (print this)

Stack: React 19 · Vite 8 · React Router 7 · axios | Node 20 · Express 5 · JWT · scrypt · multer | PostgreSQL (Supabase) · Supabase Storage | Netlify serverless | Brevo/Resend mail | node:test · ESLint

Numbers: 2 roles · 20 screens · ~40 endpoints · 9 tables · 14 chart components · 8 dashboard charts · 6 filters · 10 reports · 49/49 tests · ~28.8 k LOC · 10 files × 10 MB per application · JWT 8 h · file token 5 min

Statuses: Pending Review → Approved | Rejected; Withdrawn (student, from Pending only)

Slot rule: consumed on Approved only; returned on reversal/delete; conditional UPDATE prevents double-claim.

Duplicate rule: one pending, or one approved *this term*, per scholarship. Rejected/Withdrawn/earlier-term approval block nothing.

Eligibility weights: academic 40 · documents 30 · application standing 20 · history 10.

Renewal outlook inputs: eligibility score, GPA vs 2.00 floor, trend, document completion, past renewals; confidence ↓ with < 2 terms.

Two analytics principles: prediction ≠ decision (basis always shown) · missing input lowers confidence, never invents a value.

Security one-liners: route guards render, server authorises · identity from token never from request · uploads allow-listed + nosniff + attachment fallback · rate limits keyed by email+IP · admins never set passwords · last admin cannot be removed.

Rate limits: login 8/15 min (fails) · register 5/h · forgot 5/h · reset 10/h (fails) · verify 8/h.

Applicants ≠ applications. One renewing scholar = 1 applicant, 3 applications.

Say early: prediction not yet validated (no outcome data yet — the system creates it).